

Senior Full-Stack Developer — Take-Home Technical Assessment

- Position Level: IC4/IC5
 - Time Allocation: 4-5 hours recommended
 - Submission Requirements: All deliverables + complete AI transcript
-

Assessment Overview

This assessment evaluates your ability to design, build, and communicate full-stack solutions while leveraging AI tools as a force multiplier. You are assessed on four core performance axes:

- Impact: Measurable outcomes and business value
- Engineering Excellence: Technical depth, code quality, and best practices
- People/Leadership: Communication, documentation clarity, and influence
- Direction/Strategy: Long-term thinking, architectural vision, and scalability

Critical Requirement: You must submit the complete transcript of your AI interactions (ChatGPT, Claude, or similar). Your use of AI is evaluated as a professional skill and not penalized. Show us how you prompt, iterate, and validate AI-generated outputs.

Part 1: Recursive AI Logic & System Design

Scenario

You are building a multi-tenant project management platform where organizations can manage projects, tasks, team members, and track progress. Think of it as a simplified Linear or Jira.

Task 1.1: Data Model Design

Using an AI tool, design a GraphQL schema + PostgreSQL data model that supports:

- Multiple organizations, each with their own projects and team members
- Projects containing tasks with assignees, priorities, statuses, labels, and due dates
- Task comments and activity history (who changed what, when)
- Role-based access control (owner, admin, member, viewer per organization)
- Workspace-level settings and customizable task statuses per project

Deliverable: GraphQL type definitions + SQL CREATE TABLE statements (or ORM model definitions)

Task 1.2: Technical Documentation

Using the AI-generated schema from Task 1.1 as input, prompt your AI tool to generate comprehensive technical documentation covering:

- Data model rationale and entity relationships
- Query patterns for common operations (e.g., "get all overdue tasks assigned to me across projects")
- Indexing strategy and performance considerations
- Multi-tenancy isolation approach
- Migration strategy for evolving the schema

Deliverable: Technical documentation (2-3 pages, markdown format)

Task 1.3: AI-Driven Architecture Critique

Prompt your AI tool to perform a critical analysis of the schema you designed, specifically identifying:

- N+1 query risks in the GraphQL resolvers
- Data consistency challenges (e.g., denormalized counters, status transitions)
- Scalability bottlenecks at 100x current data volume
- Security concerns (tenant isolation, authorization gaps)
- Missing indexes or denormalization opportunities

Then provide YOUR assessment: Do you agree with the AI's critique? What did it miss? What would you prioritize fixing first and why?

Deliverable:

1. AI critique output
 2. Your written analysis (500-750 words)
-

Part 2: Full-Stack Implementation

Problem Statement

Build a Task Board feature for the project management platform described in Part 1. A Node.js GraphQL API serves the frontend.

Task 2.1: Backend — GraphQL API

Design and implement a GraphQL API that:

- Queries tasks with filtering (status, assignee, priority, due date range)
- Supports cursor-based pagination for large task lists
- Includes a summary/aggregation query (task counts by status, overdue count, tasks per assignee)
- Handles authorization (users can only see tasks in organizations they belong to)

Deliverable:

1. GraphQL schema (type definitions)
2. Resolver implementation (Node.js, minimum 80 lines of meaningful logic)
3. Brief explanation of your design decisions (200-300 words)

Task 2.2: Frontend — Task Board UI

Build a React component that:

- Consumes the GraphQL API from Task 2.1
- Displays tasks in a Kanban-style board (columns by status) or a filterable table — your choice
- Shows summary statistics (total tasks, overdue count, distribution by priority)
- Implements at least one data visualization (chart/graph)
- Handles loading, error, and empty states gracefully

Deliverable:

1. React component code (TypeScript, using hooks)
 2. Brief explanation of state management and component architecture decisions (200-300 words)
-

Part 3: Technical Communication & Product Thinking

Scenario

Your project management platform has grown to 500+ organizations. Users are reporting that the task board becomes unusably slow for projects with 2,000+ tasks. The current implementation fetches all tasks for a project in a single GraphQL query, renders the full Kanban board client-side, and re-fetches everything on every filter change or status update. Page load times are 6-8 seconds for large projects, and drag-and-drop reordering triggers a full refetch.

Task 3.1: Technical RFC

Write an internal RFC (Request for Comments) proposing a solution. It must include:

- Problem statement with data/metrics framing
- Proposed approach (at least 2 options evaluated with trade-offs)
- Recommended solution with implementation plan
- Performance targets (what "good" looks like)
- Rollout strategy (how to ship without breaking existing users)

Target Audience: Your engineering team + product manager

Deliverable: RFC document (2-3 pages, markdown)

Submission Requirements

Required Deliverables

1. Part 1:
 - Schema definitions (GraphQL + SQL/ORM)
 - Technical documentation (2-3 pages)
 - AI critique + your analysis (500-750 words)
2. Part 2:
 - Node.js GraphQL API code (schema + resolvers)
 - React component code (TypeScript)
 - Brief explanations for each
3. Part 3:
 - RFC document (2-3 pages)
4. AI Transcript:
 - Complete, unedited transcript of all AI interactions during this assessment
 - Include all prompts, iterations, and refinements
 - Add brief commentary on your prompting strategy

Submission Format

- Single GitHub repository (preferred) or ZIP file
- Organized directory structure with clear README
- All documentation in markdown format
- Code properly formatted with comments
- Include a `package.json` so we can run your code

Time Management Guidance

- Part 1 (System Design + AI Critique): 90-120 minutes
- Part 2 (Implementation): 90-120 minutes
- Part 3 (RFC): 60-90 minutes
- Documentation & cleanup: 30 minutes

Total: 4-5 hours

Questions?

If you have clarifying questions about the assessment, please email [kcmanikanta@compliancefoundry.ai]. We encourage you to make reasonable assumptions when specifications are ambiguous, and to document them in your submission.

Deadline: 7 days from assignment

This assessment evaluates real-world engineering skills. We value practical expertise, clear thinking, and effective use of modern AI tools to amplify your capabilities.